If you have existing tests on User in that situation, those tests should continue to pass.

Normally I don't change tests when I refactor—the goal is to test functionality, not implementation. I will sometimes make an exception to this rule if I move the code to a new class, especially if I expect the new class to be shared often.

When you break out related attributes into their own class, as in this Name example, you'll often find it's much easier to add complexity when you have a dedicated place for that logic. When you need middle names or titles, it's easier to manage those in a separate class than it would be if you had a half implementation of names in multiple classes.

You'll also find that these small classes are easy to test because Name no longer has a dependency on the database or any other code. Without dependencies, it's easy to set up and write fast tests for name logic:

```
it "generates sortable names" do
  name = Name.new("Noel", "Rappin")
  expect(name.sort_name).to eq("Rappin, Noel")
end
```

The tests for the new Name class are quick to write and to run. The easier it is to write tests, the more tests you'll write.

Also look out for repeated if statements or other conditionals that switch on the same values. A common example is continually checking for a nil value. Another is frequent checking against a status variable. The task tracker, for example, might have a lot of methods that do this:

```
if status == :completed
  calculate_completed_time
else
  calculate_incompleted_time
end
```

Every program needs the occasional if statement. But if you're continually checking an object's state to determine what to do next, consider the possibility that you have a new set of classes. For example, the preceding snippet implies the existence of something like CompleteTask and IncompleteTask. (Or maybe completeness and incompleteness affect only part of the class functionality, so you get something like CompleteTaskCalculator and IncompleteTaskCalculator.)

Once you've separated functionality into separate classes, an object-oriented program is supposed to switch based on class, using message passing and polymorphism: